CS1783 Artificial Intelligence Programming

Fully Connected Neural Networks

Jacob Shultz

Overview

In this assignment I am tasked with training two neural networks: one to predict the price of a car and another to predict whether a mushroom is edible or poisonous.

Explaining Parameters

I will tweak the following parameters to fine-tune my models

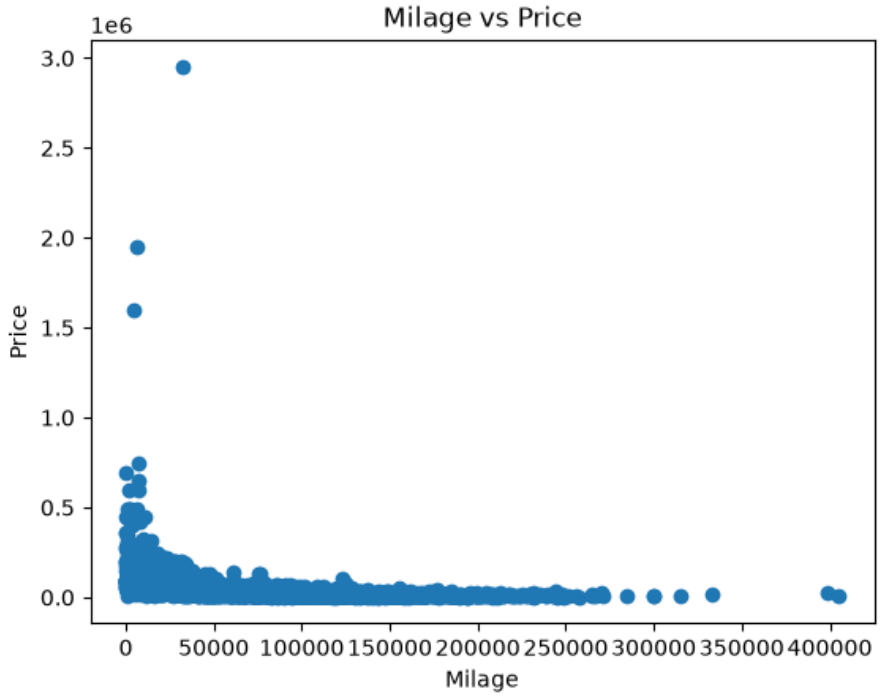
- Layers: A “deep” network (layers) captures hierarchical relationships more effectively. A “wide” network (nodes per layer) captures intricacy.
- Activation: The function that limits the firing of each neuron.
- Loss: The function used to calculate how wrong each neuron is.
- Optimizer: The function used to adjust weights and biases.
- Learning Rate: A parameter of the optimizer, controls how dramatically each adjustment is made.
- Epochs: The number of times the data is passed over during training. One epoch is one complete pass through the data.
- Batch Size: The number of rows/samples processed in between optimizer updates.
- Dropout: (weight decay) the rate that weights “die,” makes the optimizer penalize large weights.

Car Prices (Regression)

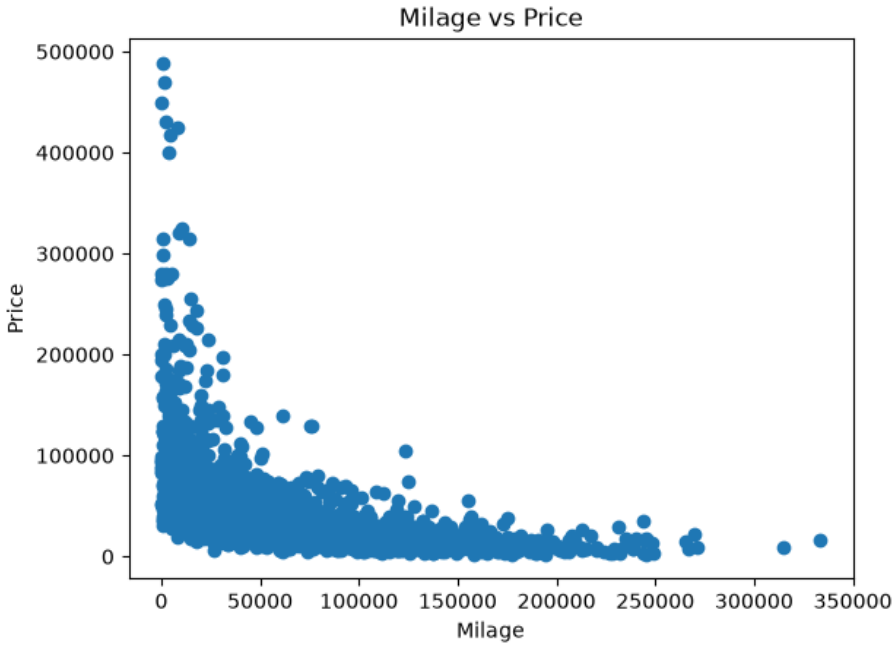
Data Cleaning

This data was extremely messy, so I began by thoroughly cleaning. For the majority of columns a simple one-hot encoding limited by a minimum frequency sufficed. However, the engine and transmission features were strings comprised of multiple sub-values. These required regex-ing out and appending the values in new columns. After all this I was left with 84 features.

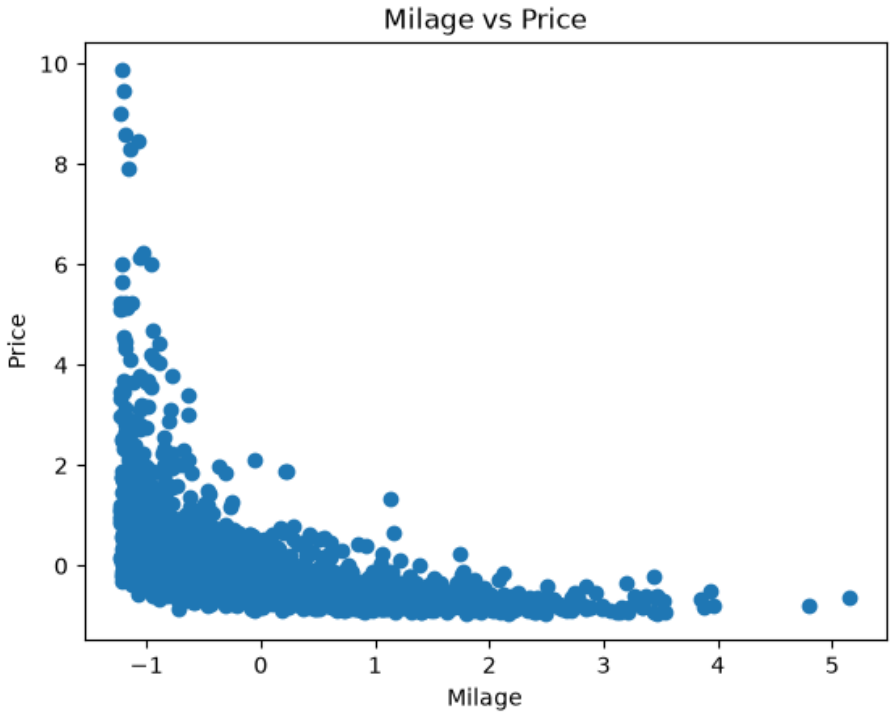
I noticed that there were some pretty hateful outliers in the data, which, in initial runs, were causing exploding gradients.



I just opted to remove any rows which had outliers (not within 6 standard deviations from the mean) in any column.



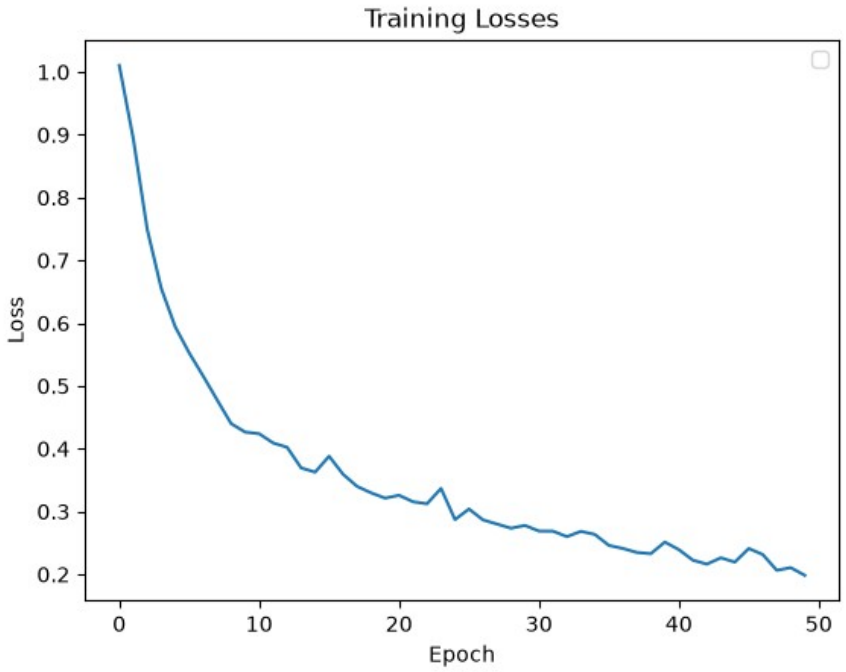
I then scaled the data to further prevent issues with exploding gradients.



Trade offs were made, but this should be good enough for Broadway.

Initial Results

Layers	84 input, halving each layer down to 1 neuron in the output. 5 layers total.
Activation	ReLU
Loss	MSE
Optimizer	Adam
Learning Rate	0.001
Epochs	50
Batch Size	64
Dropout	0.0
Final Train Loss	0.2
Test Loss	0.29 ($12,511)

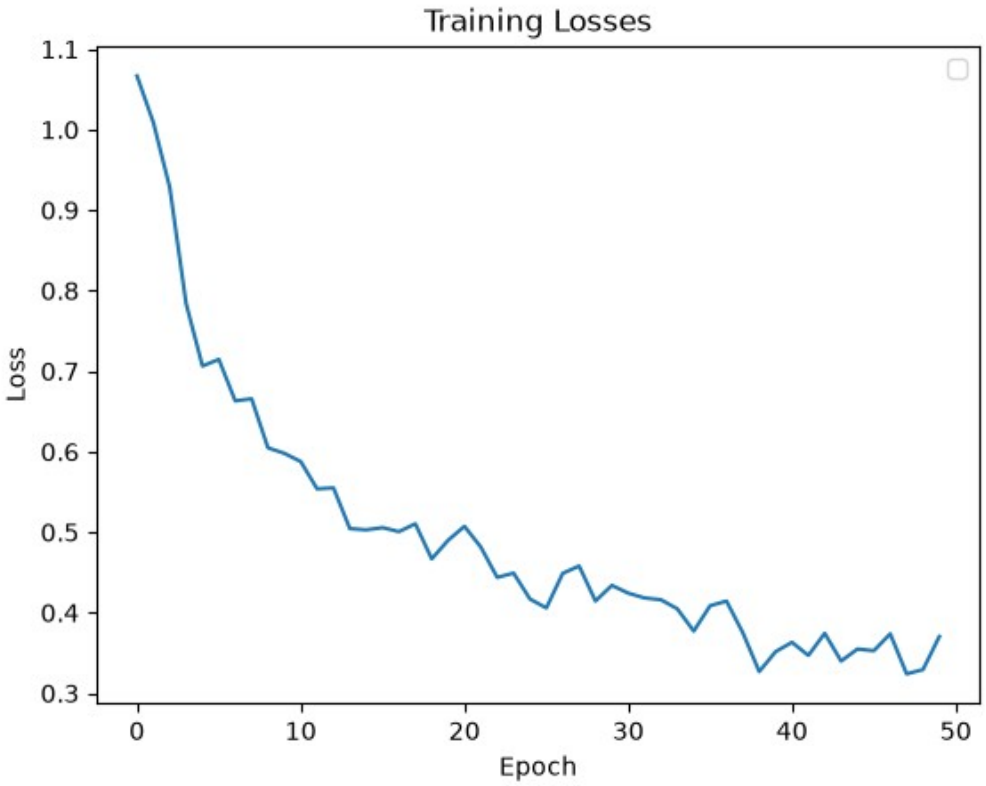


Thanks to all the cleaning the loss is out of the billions (trillions?). A test loss higher than our train loss may suggest some degree of over-fitting is happening. The training loss graph looks good; sharp decline followed by the leveling-off suggests we already have enough neurons to encapsulate the data. Being off by 12.5 grand isn't great but it isn't a terrible starting point.

Fixing Over fitting

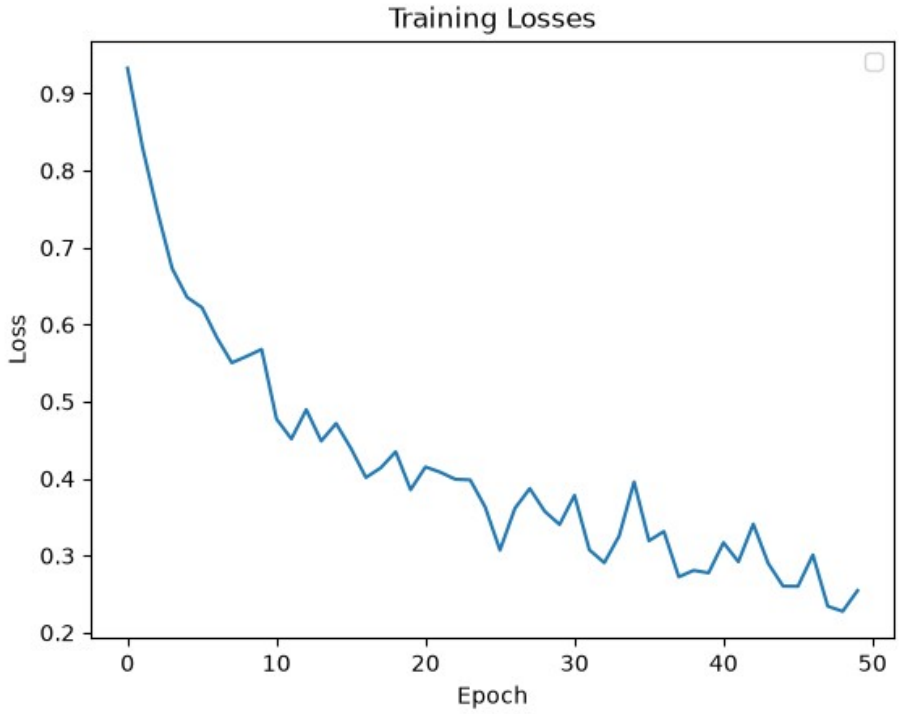
Let's add a dropout rate of 0.3 to all our hidden layers.

Layers	84 input, halving each layer down to 1 neuron in the output. 5 layers total.
Activation	ReLU
Loss	MSE
Optimizer	Adam
Learning Rate	0.001
Epochs	50
Batch Size	64
Dropout	0.3
Final Train Loss	0.37
Test Loss	0.34 ($13,846)



Our errors are now closer to one another although they got higher. I'll beef up the layers to see if that helps draw the error down further.

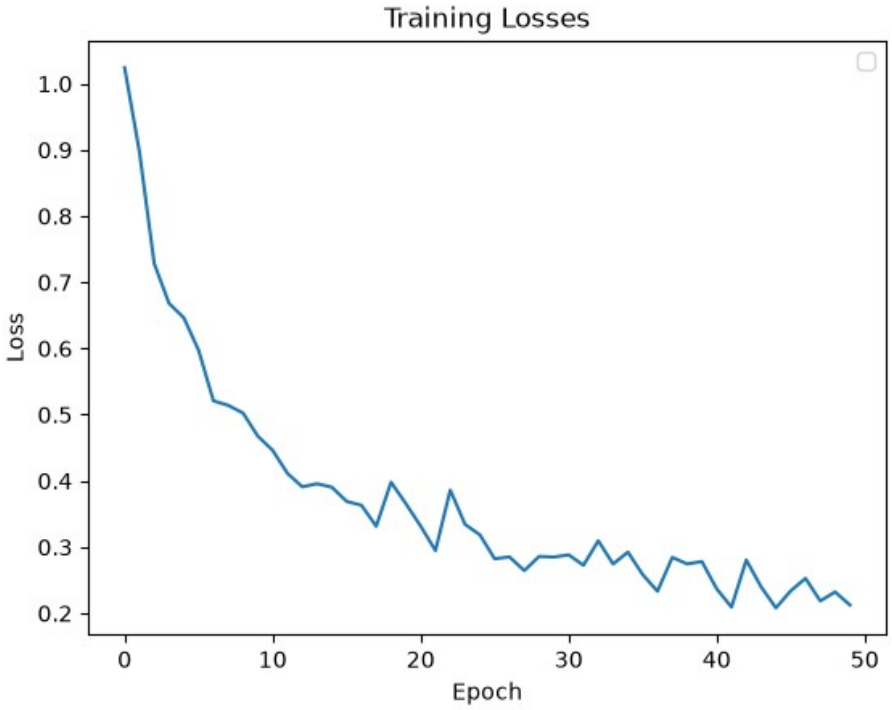
Layers	84 → 100 → 80 → 60 → 40 → 20 → 10 → 5 → 1
Activation	ReLU
Loss	MSE
Optimizer	Adam
Learning Rate	0.001
Epochs	50
Batch Size	64
Dropout	0.3
Final Train Loss	0.25
Test Loss	0.45 ($12,781)



Well, our test MSE loss went up but the loss in dollars went down. This is probably due to the dollar loss computation being based on absolute values. Later I want to switch the error to MAE, but let's fix one issue at a time.

The over-fitting problem got worse. I wonder if having such narrow layers leading up to the output is leading to some of this, choking the output.

Layers	84 → 100 → 80 → 60 → 40 → 20 → 1
Activation	ReLU
Loss	MSE
Optimizer	Adam
Learning Rate	0.001
Epochs	50
Batch Size	64
Dropout	0.3
Final Train Loss	0.21
Test Loss	0.19 ($10,965)

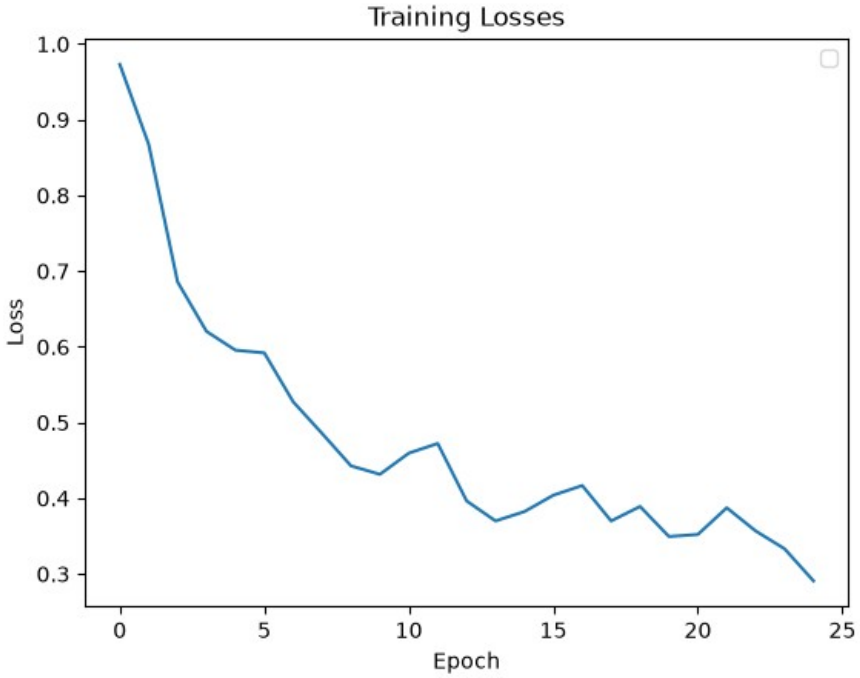


Great! Our best run yet. I'm still not happy with that $11k error. Let's do some other experiments to see if we can find a better way to go about this.

Experimentation

Since beefing up the last layers worked, I'll try beefing up all the layers. I'll also increase the dropout rate and lower epochs to prevent over fitting.

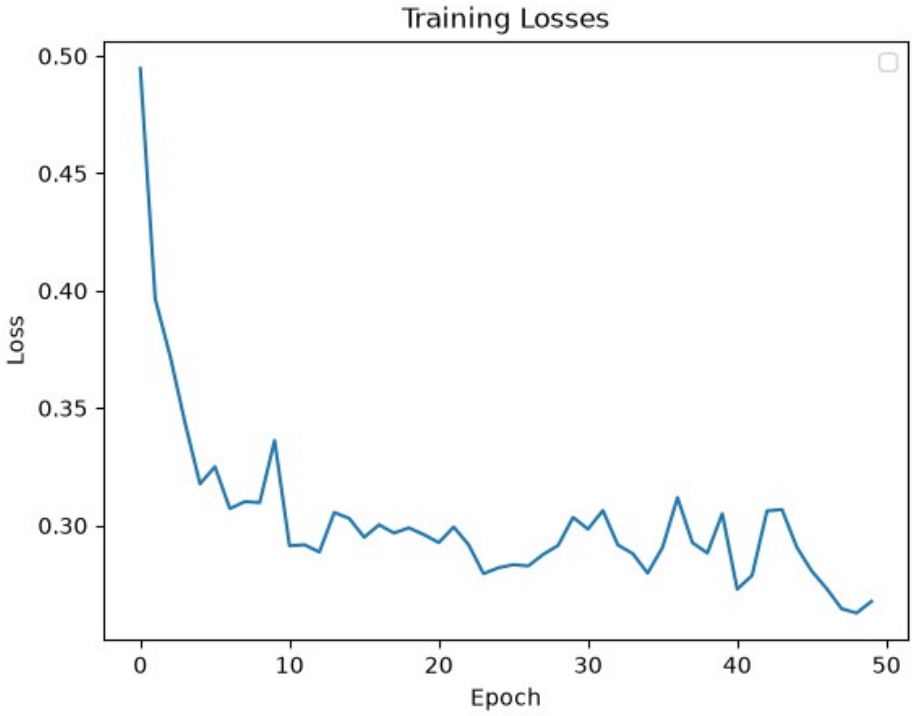
Layers	84 → 100 → 100 → 80 → 60 → 40 → 30 → 20 → 1
Activation	ReLU
Loss	MSE
Optimizer	Adam
Learning Rate	0.001
Epochs	25
Batch Size	64
Dropout	0.5
Final Train Loss	0.33
Test Loss	0.37 ($14,711)



Well, that just made it worse. I think we just can't train enough without over fitting. Just take my word for it; increasing the epochs here causes it to over fit dramatically.

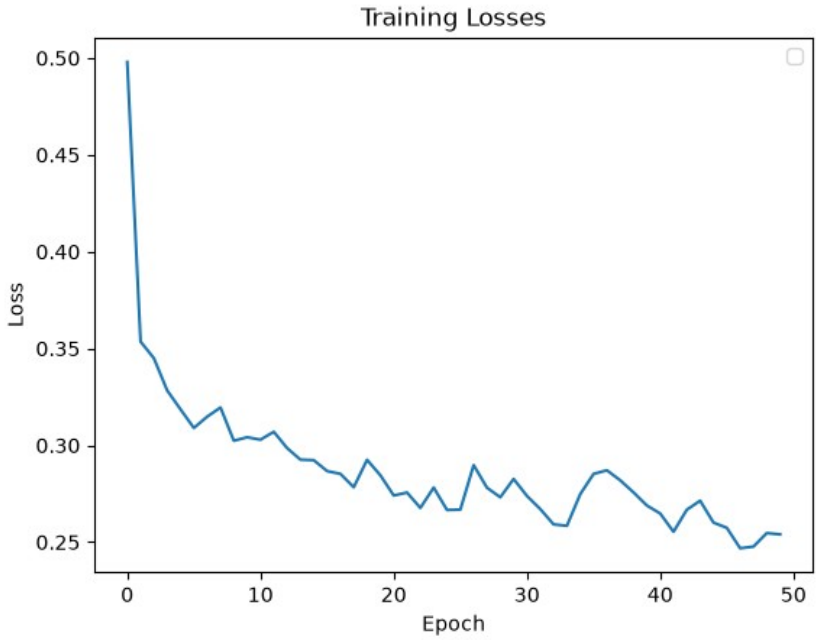
Trying something completely different: using MAE. MAE produces smaller gradients which makes optimization happen slower. So I set dropout to 0.2, epochs to 50, and the learning rate to 0.01. This actually produces a decent (comparatively) result. We got spot-on generalization with equal train and test losses. I think the different error function makes quite a bit of difference in a larger network.

Layers	84 → 100 → 100 → 80 → 60 → 40 → 30 → 20 → 1
Activation	ReLU
Loss	MAE
Optimizer	Adam
Learning Rate	0.01
Epochs	50
Batch Size	64
Dropout	0.2
Final Train Loss	0.27
Test Loss	0.27 ($11,992)



Lets try using MAE on a smaller size network.

Layers	84 → 100 → 80 → 60 → 40 → 20 → 1
Activation	ReLU
Loss	MAE
Optimizer	Adam
Learning Rate	0.01
Epochs	50
Batch Size	64
Dropout	0.2
Final Train Loss	0.25
Test Loss	0.22 ($9,925)

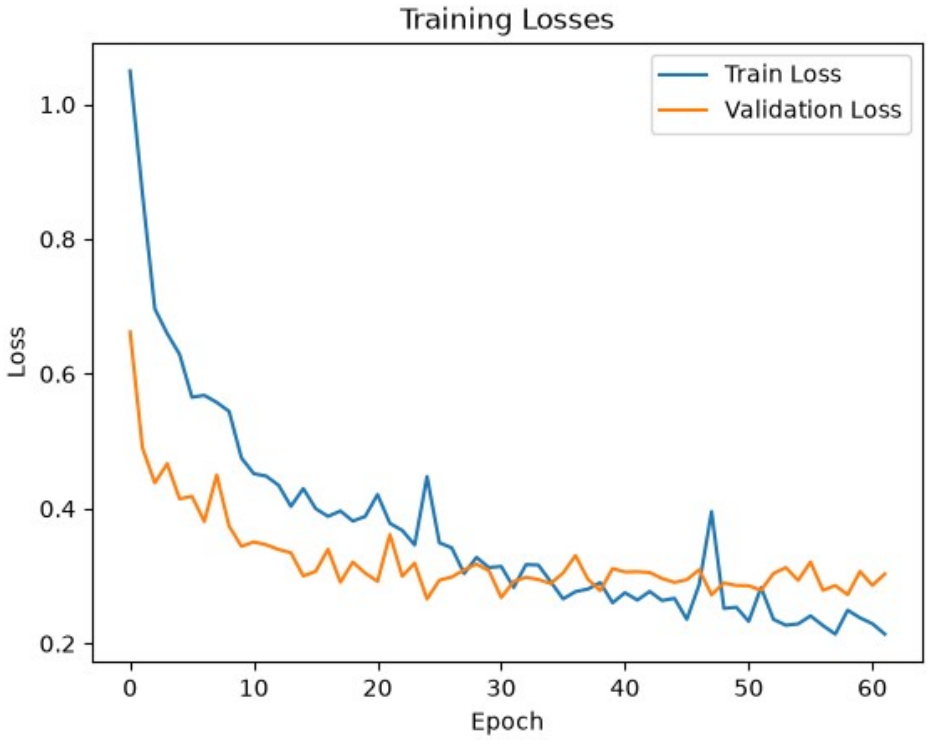


Excellent. This is our best result yet. It seems like MAE with a smaller net is the way to go here.

To end my experimentation I need to mess with early stopping. I'll need to modify my training loop to calculate validation per epoch rather than right at the end. I'll also add a new parameter: Patience, which specifies how many epochs in a row validation loss can be higher than training loss before the loop terminates. One more thing: I now store the best model trained during the loop.

Let's test this new functionality on the model which was over fitting.

Layers	84 → 100 → 100 → 80 → 60 → 40 → 30 → 20 → 1
Activation	ReLU
Loss	MSE
Optimizer	Adam
Learning Rate	0.001
Epochs	100
Batch Size	64
Dropout	0.3
Patience	10
Final Epoch	62
Final Train Loss	0.21
Best Model Loss	0.27 ($12,742)



The mechanism definitely worked. We managed to stop the model right as the losses converged. This still didn't yield a superb result but this performed better than the run we had before of this model in which it really, really over fit.

This now will end my experimentation. I am going to tinker a bit more off paper until I feel I've produced the best possible model to my ability.

Final Model for Car prices

My final model has taken all the results from experimentation and incorporated them to get a model which I believe is of my best effort.

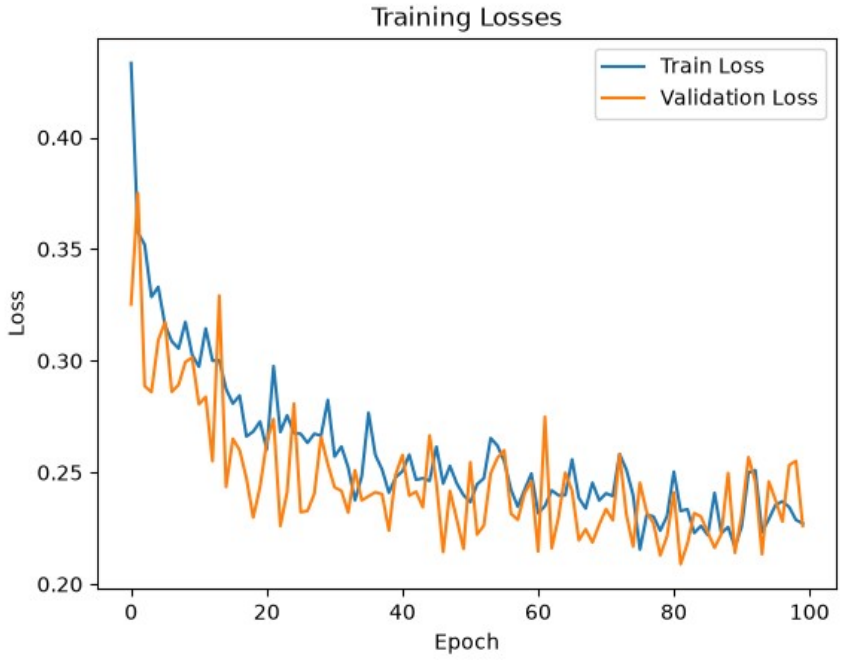
The parameters:

Layers	85 → 70 → 55 → 35 → 20 → 1
Activation	ReLU
Loss	MAE
Optimizer	Adam
Learning Rate	0.01
Epochs	100
Batch Size	64
Dropout	0.2

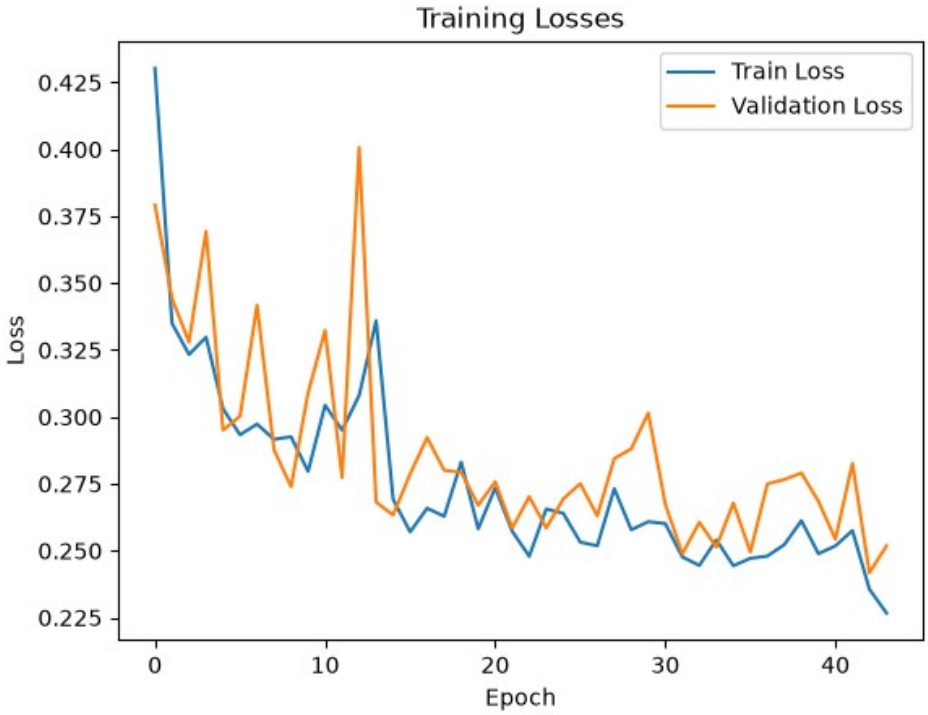
Patience	10

And, the results:

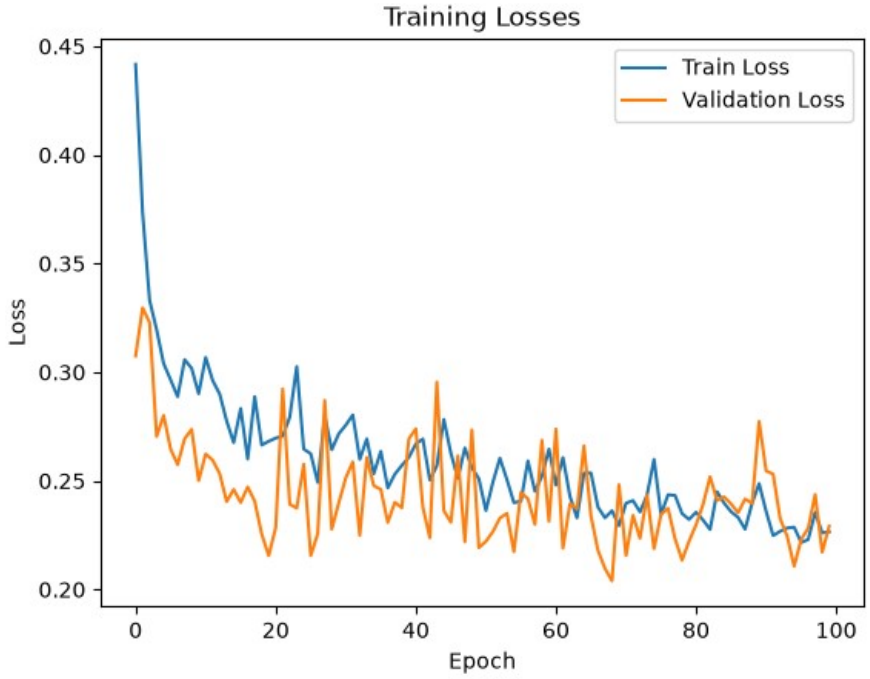
Final Epoch	100
Final Train Loss	0.23
Best Model Loss	0.21 ($9,382)



Final Epoch	44
Final Train Loss	0.22
Best Model Loss	0.24 ($10,856)



Final Epoch	100
Final Train Loss	0.23
Best Model Loss	0.2 ($9,158)



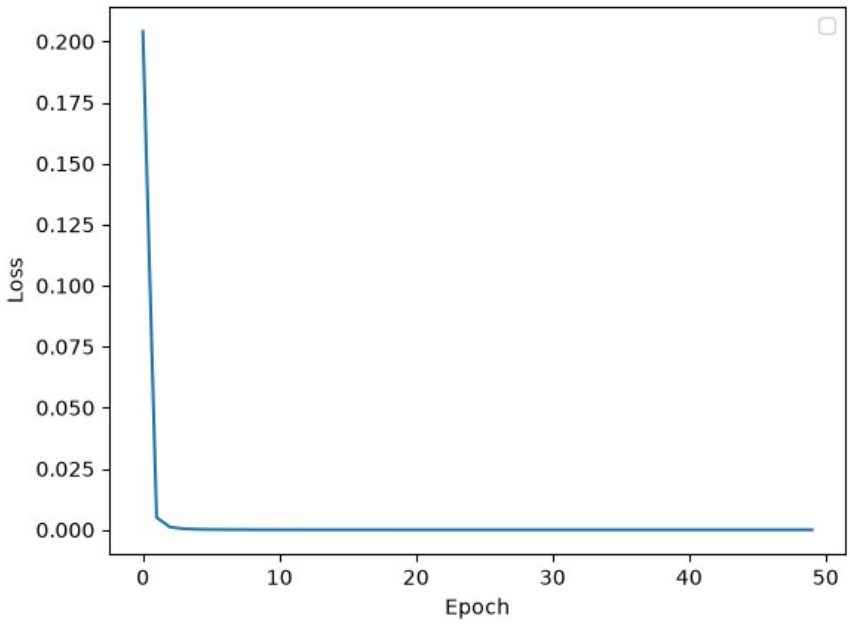
Mushrooms (Classification)

Data Cleaning

Not much to say for this data set. I simply cleaned up the null strings (“?”) and one-hot encoded the whole thing. In total I have 116 features and 2 output classification columns.

Initial Results / Experimentation

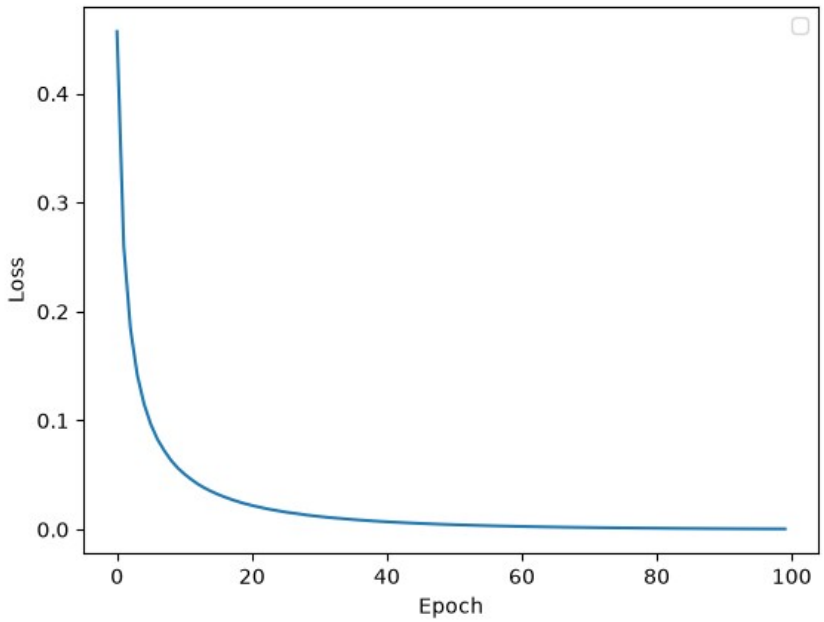
Layers	116 → 100 → 50 → 2
Activation	ReLU
Loss	Cross Entropy
Optimizer	Adam
Learning Rate	0.001
Epochs	50
Batch Size	64
Dropout	0.0
Final Train Loss	0.0
Test Loss	0.0
Accuracy	100%



Wow, looks like we already have a perfect model. This is awkward! This is likely due to this problem being able to be solved with pure memorization. We have a finite of mushroom combinations with a deterministic classification for each combination. Regardless, let’s do some experiments.

Let’s see how small of a model we can make. Who needs hidden layers anyways?

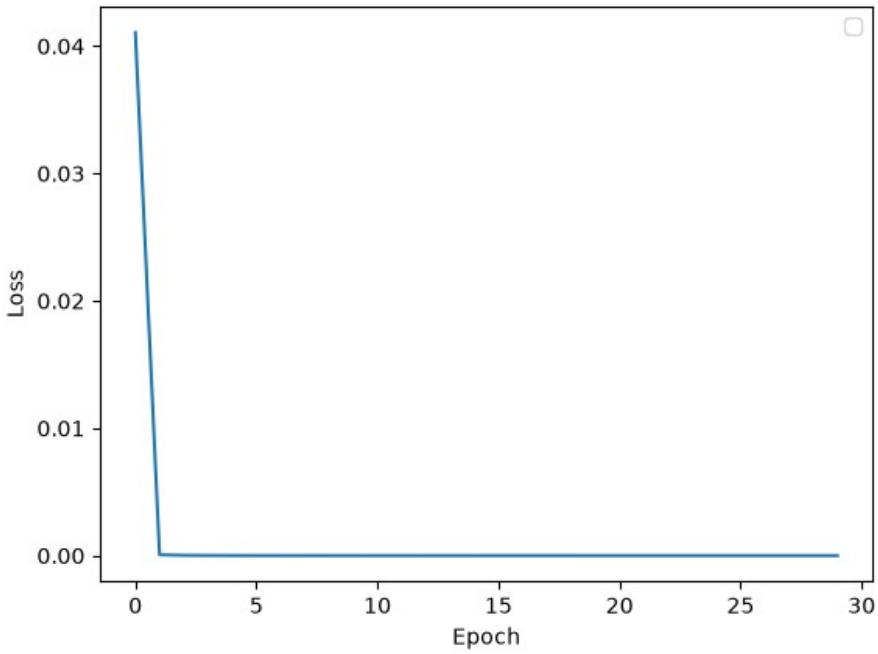
Layers	116 → 2
Activation	ReLU
Loss	Cross Entropy
Optimizer	Adam
Learning Rate	0.001
Epochs	100
Batch Size	64
Dropout	0.0
Final Train Loss	0.0
Test Loss	0.0
Accuracy	100%



Is it possible to make a model that over fits?

Layers	116 → 1000 → 1000 → 2
Activation	ReLU
Loss	Cross Entropy
Optimizer	Adam
Learning Rate	0.001
Epochs	30

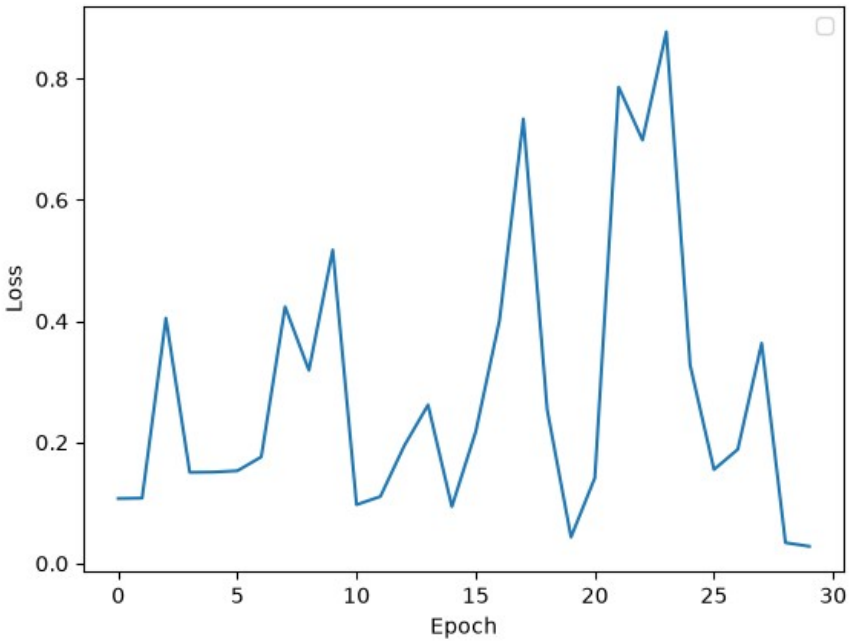
Batch Size	64
Dropout	0.0
Final Train Loss	0.0
Test Loss	0.0
Accuracy	100%



Doesn’t look like it.

What if we make a model with too much “noise,” a high dropout rate and a high learning rate?

Layers	116 → 1000 → 1000 → 2
Activation	ReLU
Loss	Cross Entropy
Optimizer	Adam
Learning Rate	0.01
Epochs	30
Batch Size	64
Dropout	0.7
Final Train Loss	0.03
Test Loss	0.0
Accuracy	100%



About what I expected: high dropout + high learning rate + lots of neurons means a network that is constantly over-correcting. We still managed to get a perfect model right at the end!

Final Model for Mushroom Edibility

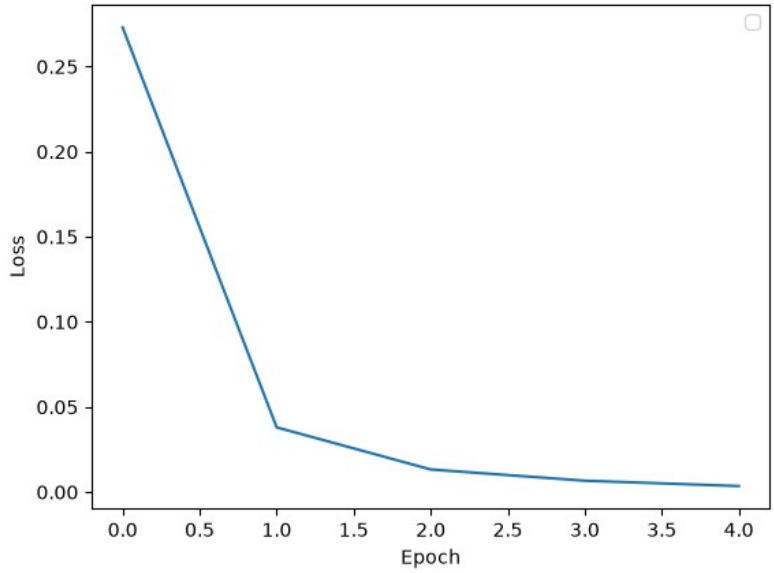
My final model has been engineered for efficiency. We use a single hidden layer to minimize the number of epochs required, allowing the fastest training time. The model is so small that training seems to be deterministic as demonstrated by the graphs.

Parameters:

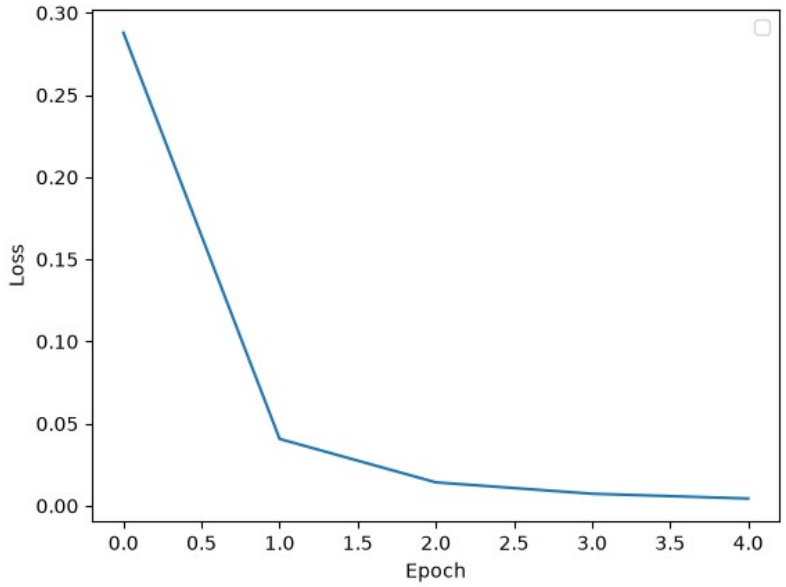
Layers	116 → 50 → 2
Activation	ReLU
Loss	Cross Entropy
Optimizer	Adam
Learning Rate	0.01
Epochs	5
Batch Size	64
Dropout	0.0

Results:

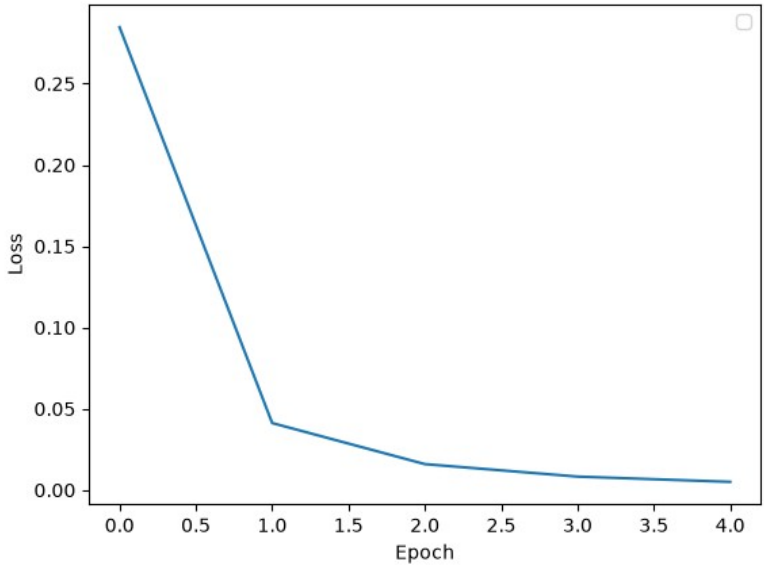
Final Train Loss	0.0
Test Loss	0.0
Accuracy	100%



Final Train Loss	0.0
Test Loss	0.0
Accuracy	100%



Final Train Loss	0.0
Test Loss	0.0
Accuracy	100%


Loss
0.25
0.20
0.15
0.10
0.05
0.00
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch